

Part A: Neural Machine Translation

In this lab, a sequence-to-sequence (seq2seq) Neural Machine Translation model was built to translate Vietnamese into English. The lab began with a basic encoder-decoder architecture, which was then upgraded with a Luong attention mechanism to improve translation performance.

Task 1: Implementing the Encoder

For this task, the foundational layers for the model were set up to process the vocabulary, in other words, implement the encoder.

Two embedding layers (one for source and one for target) were initialized to convert word indices into dense vectors, ensuring that padding tokens were ignored. Next, an LSTM layer was created to process the source sentence.

In the forward pass, dropout was applied to the embeddings, which were then passed through the LSTM to encode the source sequence and decode the target sequence.

Task 2: Implementing the Decoder and the Inference Loop

The inference loop, used to generate translations one word at a time, was implemented for the evaluation phase. Unlike the training phase, the inference decoder processes one token per step. It receives the previous step's hidden states along with the embedded current token. Therefore, this is a basic model that relies solely on the final hidden state of the encoder, causing it to lose context and forget earlier parts of long sentences.

Consequently, the resulting BLEU score on the test set was 4.22, which is exceptionally low.

Task 3: Adding Attention

The model was then upgraded by adding an Attention Mechanism. An attention score was calculated using the dot product between the decoder's current state and every hidden state from the encoder. This modification allows the model to dynamically focus on the most relevant parts of the original sentence at every generation step.

It drastically improves the translation performance, obtaining a BLEU score on the test set of 15.19, compared to the previous 4.22.

Sample Translations:

Sample 1:

- **Reference:** the second quote is from the head of the u.k. financial services <unk> .

- **Basic Candidate:** the <unk> <unk> <unk> , <unk> , <unk> , <unk> , <unk> .
- **Attention Candidate:** <unk> second <unk> from the <unk> <unk> <unk> <unk> <unk> <unk>

Sample 2:

- **Reference:** it gets worse .
- **Basic Candidate:** the <unk> <unk> <unk> , <unk> <unk> , <unk> <unk> .
- **Attention Candidate:** it 's worse .

While both models output some unknown tokens (<unk>), the basic model suffers from severe repetition, which attention improves.

PART B: Using the Pre-trained BERT models

In this lab, DistilBERT is used to perform Aspect-Based Sentiment Analysis (ABSA). Unlike general sentiment analysis, ABSA focuses on the sentiment toward a specific aspect (target) within a sentence. Three different model approaches have been studied: Sequence Classification, Bag-of-Words (Global Average Pooling) and LSTM.

Task 1: Data pre-processing

The goal of this task is to transform raw XML review data into a format compatible with the DistilBERT architecture. Aspect-Based Sentiment Analysis (ABSA) requires the model to consider both the context (sentence) and the target (aspect).

Using the tokenizer.encode_plus method, the review text and the aspect term are concatenated. This automatically inserts the essential [SEP] token between them, creating a standardized sequence: [CLS] Sentence [SEP] Aspect [SEP].

To make the text readable for the model, words are converted into Input IDs, which act as index numbers from the DistilBERT vocabulary. Since the model requires every input to be the same length (128 tokens), padding is applied to fill empty spaces in shorter sentences with zeros. Finally, an Attention Mask is generated so the model distinguishes between word tokens and the empty padding that it should ignore.

Task 2: Basic classifiers using BERT: Model 1 and Model 2

Model Architecture:

- **Model 1(Prebuilt Sequence Classification):** Uses the pre-trained [CLS] token output. This token is designed to capture the semantic summary of the entire sentence.

- Model 2 (Neural Bag-of-Words): Uses Global Average Pooling to average all word vectors in the sentence, treating the input as an unordered collection of words.

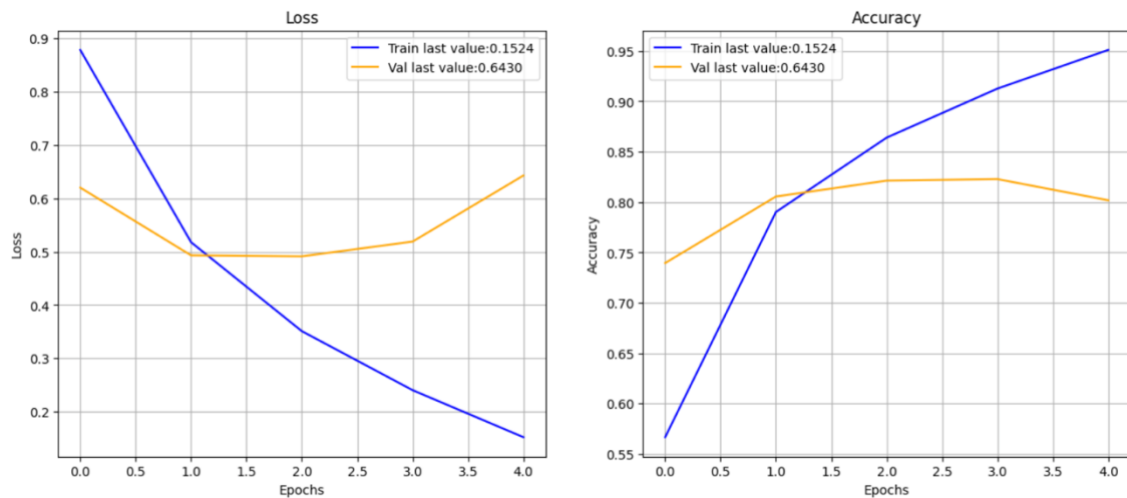


Figure 1: Loss and Accuracy curves Model 1 (5 epochs)

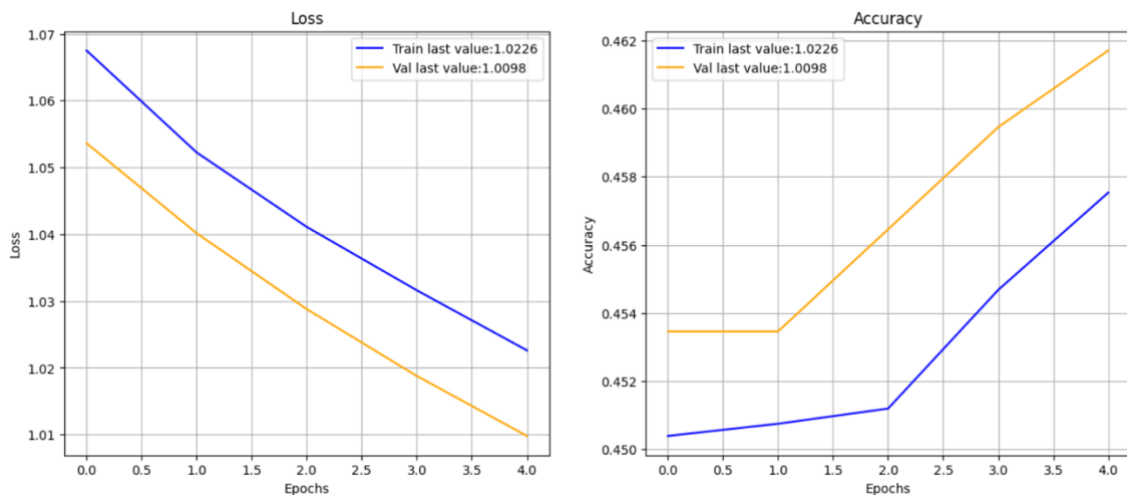


Figure 2: Loss and Accuracy curves Model 2 (5 epochs)

Performance comparison:

Model 1 achieves a test accuracy of 81.59%. Indicating a high performance. Additionally, it converges quickly by the second and third epoch. On the other hand, Model 2 achieves a test accuracy of 46.48%, extremely low.

Therefore, Model 1 significantly outperforms Model 2. It is expected as Model 1 fine-tunes the entire BERT relationship using the specialized [CLS] token that captures all the semantic information over the entire sequence. Model 2, because it averages all word vectors, loses the specific relationship between the aspect and the sentiment, making it harder for the model to distinguish.

Task 3: Advanced classifier using BERT: Model 3.

Instead of simple averaging like Model 2, BERT embeddings are fed into an LSTM layer. The LSTM processes the sequence step by step, allowing the model to capture the order of the words and the specific context surrounding the aspect.

Its accuracy on the test set is higher than Model 2, but still performing poorly with a 52.84%.

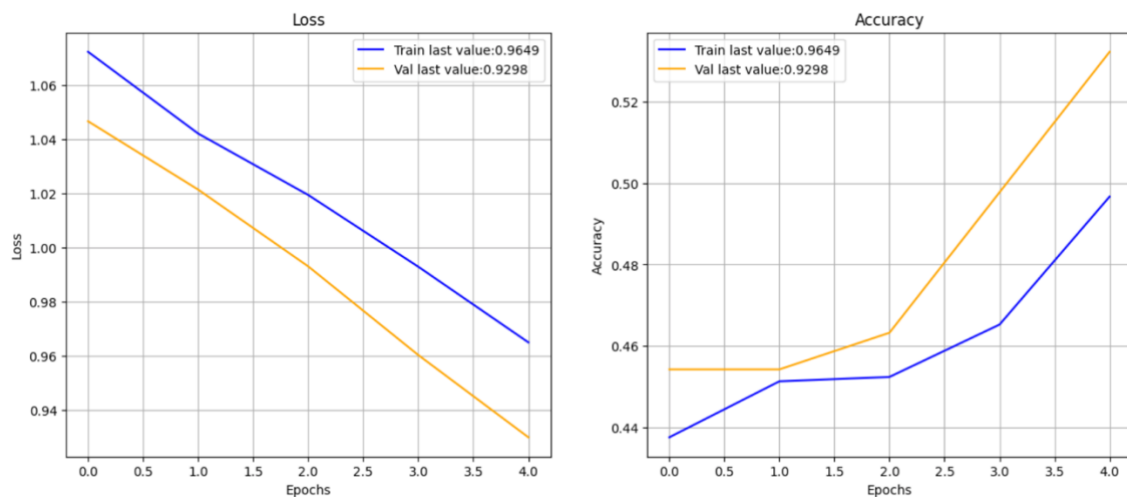


Figure 3: Loss and Accuracy curves Model 3 (5 epochs)

The results show that the standard Sequence Classification (Model 1) is the most effective for ABSA.

PART C: Natural Language Generation

Task 1: Build and evaluate greedy and beam search algorithms from scratch.

Greedy:

- Output:

The cat slept on the floor, and the cat was still asleep.

"I was just trying to get her to sleep," she said. "I was trying to get her to sleep, but she was still asleep."

The cat

- Log-probability: -62.58576202392578

Beam Search:

- Output:

The cat slept on the floor of the house.

"It was like a nightmare," she said.

"It was like a nightmare. It was like a nightmare. It was like a nightmare. It was like a nightmare. It

- Log-probability: -46.066965823905775

Beam Search achieved a higher log-probability than Greedy Search. This is expected as Greedy Search only selects the locally optimal token at each step. It is short-sighted and cannot backtrack if a high-probability word leads to a low-probability sentence later. On the other hand, Beam Search maintains multiple paths ($k=3$). It can choose a word that is less likely in the moment if it leads to a sequence with a higher cumulative probability.

Nevertheless, both algorithms suffered from a repetitive loop.

- Greedy looped "still asleep."
- Beam Search looped "It was like a nightmare."

In Beam Search, repetition is common because a simple, repeating phrase often has a higher cumulative probability than a complex, unique sentence. Without a repetition penalty, the algorithm optimizes for the safest, most likely sequence, which is often a loop.

Task 2: Build and evaluate the beam search algorithm with a repetition penalty.

Beam search with a repetition penalty:

- **Output:**

The cat slept on the floor of her room, and when she woke up to find that it had been eaten by a large black cat in the middle of her room, she immediately ran to help. She had no idea what was going on with the

- **Log-probability:** -77.42537941224873

Beam search without a repetition penalty was free to choose the absolute most likely tokens. As a result, the model found that repeating "It was like a nightmare" had very high individual token probabilities, which led to a repetitive loop.

With a repetition penalty, the most likely tokens were manually suppressed by identifying previously used words and assigning them a near-zero probability of -1×10^9 . This forces the model to select sub-optimal candidates, rather than repeating itself. Although this modification results in a lower log-probability (from -46.06 to -77.42), it produces text that is more coherent. This demonstrates that for language models, the sequence with the highest cumulative probability is not always the most linguistically natural one.

Task 3: Experiment and assess sampling results in terms of coherence and diversity.

Random Sampling:

- **output:**

*The cat slept on the floor of her room, while she held it to her chest.
When Kiel went to check on him, a nurse found that Ejima had taken her cat.
Kiel told the nurse that she*

Random sampling produces a coherent narrative with grammatically correct sentences. However, it introduces characters (Kiel and Ejima) without providing context. Nevertheless, the diversity is high, unlike greedy or standard beam search, this method avoids repetitive loops.

High temperature:

- **output:**

The cat slept on the walls. Two kids lay here — the boys went here with two very quiet girls and one with one pretty girl (for two months they hadn't got a boy since that day's story): one was not a parent and three

Its coherence is significantly low. Cats can't sleep on walls, and the sentences don't make sense overall. On the other hand, the diversity is extremely high. The high temperature forces the model to choose very unlikely words, creating a lot of variety at the expense of logical flow.

Low temperature:

- **output:**

*The cat slept on the floor, and the cat was still asleep.
The cat was not the only one who had been affected.
The cat was also found to have been in a coma.
The cat had been found to*

In this case, the coherence is extremely high. The sentences are well-structured, concise, and convey a clear meaning. Nevertheless, the diversity is very low. The model becomes repetitive, starting every sentence with "the cat". With low temperature, the model sticks to the safest words, becoming very repetitive.

Top p:

- **output:**

The cat slept on the door to his basement as a friend found it. That wasn't to say it didn't hurt, as the couple grew to love each other; in fact, while they probably already knew it had been broken, and that you

This method generates a more complex and natural story involving the cat, a couple, and a friend. While similar to random sampling, the transitions feel more connected. It has a high level of diversity but still maintains narrative coherence.

PART D: Social Media Processing

Task 1: Build and evaluate two regression models for humour rating prediction (a) without special pre-processing applied and (b) with special pre-processing applied to the training data.

Model A without special pre-processing:

- **Model structure:**

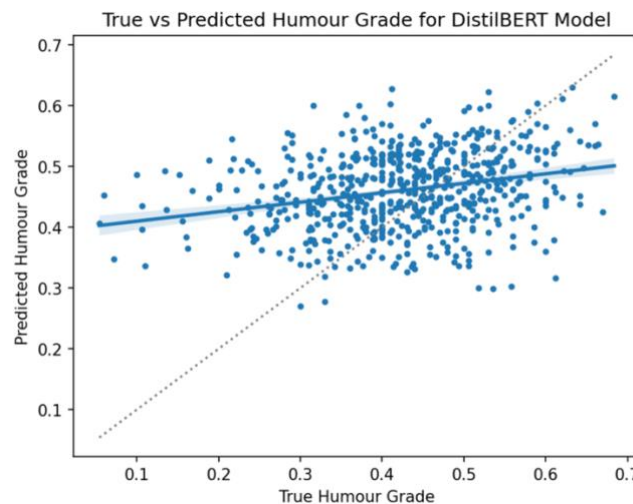
```

BERTRegressionModel(
  (bert): DistilBertModel(
    (embeddings): Embeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (transformer): Transformer(
      (layer): ModuleList(
        (0-5): 6 x TransformerBlock(
          (attention): DistilBertSelfAttention(
            (q_lin): Linear(in_features=768, out_features=768, bias=True)
            (k_lin): Linear(in_features=768, out_features=768, bias=True)
            (v_lin): Linear(in_features=768, out_features=768, bias=True)
            (out_lin): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (sa_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (ffn): FFN(
            (dropout): Dropout(p=0.1, inplace=False)
            (lin1): Linear(in_features=768, out_features=3072, bias=True)
            (lin2): Linear(in_features=3072, out_features=768, bias=True)
            (activation): GELUActivation()
          )
          (output_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        )
      )
    )
    (global_avg_pool): GlobalAveragePooling1DMasked()
    (hidden_layer): Linear(in_features=768, out_features=16, bias=True)
    (activation): Sigmoid()
    (output_reg): Linear(in_features=16, out_features=1, bias=True)
  )
)

```

The model architecture consists of the pre-trained DistilBERT base, followed by a custom masked global average pooling layer, a dense hidden layer (16 units), and a final regression output layer activated by a Sigmoid function.

- **True vs Predicted Humour Grade for DistilBERT Model scatter plot:**



The scatter plot for Model A reveals that the model struggles to accurately predict the full range of humour grades. The ideal perfect-prediction line is represented by the dotted grey diagonal line. However, the model's actual predictions (the solid blue regression line) are significantly flatter. Additionally, the predictions are heavily clustered horizontally around the mean score (approx. 0.45). It indicates the model minimizes its mean squared error by constantly guessing close to the average rating.

Results:

- **Training results:**

Epoch 1/5: Train Loss: 0.0124, Val Loss: 0.0113

Epoch 2/5: Train Loss: 0.0108, Val Loss: 0.0106

Epoch 3/5: Train Loss: 0.0096, Val Loss: 0.0102

Epoch 4/5: Train Loss: 0.0085, Val Loss: 0.0103

Epoch 5/5: Train Loss: 0.0072, Val Loss: 0.0106

- **Test loss:** 0.0130

- **Test Mean Squared Error:** 0.0136

Model B with special pre-processing (using the Ekphrasis library):

Results:

- **Training results:**

Epoch 1/5: Train Loss: 0.0133, Val Loss: 0.0119

Epoch 2/5: Train Loss: 0.0113, Val Loss: 0.0113

Epoch 3/5: Train Loss: 0.0105, Val Loss: 0.0107

Epoch 4/5: Train Loss: 0.0096, Val Loss: 0.0107

Epoch 5/5: Train Loss: 0.0087, Val Loss: 0.0105

- **Test loss:** 0.0127

- **Test Mean Squared Error:** 0.0131

Performance comparison (Model A vs Model B):

Special preprocessing does not drastically change the performance for this task.

The Test MSE slightly dropped from 0.0136 to 0.0131.

Task 2: Augment the training data twice for the regression model (Task 1 above) with the two following methods (a) synonym replacement from Wordnet and (b) deletion of random words.

Method A (synonym replacement from Wordnet):

- **Training results:**

Epoch 1/5: Train Loss: 0.0123, Val Loss: 0.0112

Epoch 2/5: Train Loss: 0.0105, Val Loss: 0.0104

Epoch 3/5: Train Loss: 0.0088, Val Loss: 0.0105

Epoch 4/5: Train Loss: 0.0068, Val Loss: 0.0107

Epoch 5/5: Train Loss: 0.0049, Val Loss: 0.0115

- **Test loss:** 0.0126

- **Test Mean Squared Error:** 0.0129

Method B (deletion of random words):

- **Training results:**

Epoch 1/5: Train Loss: 0.0122, Val Loss: 0.0112

Epoch 2/5: Train Loss: 0.0103, Val Loss: 0.0106

Epoch 3/5: Train Loss: 0.0090, Val Loss: 0.0107

Epoch 4/5: Train Loss: 0.0070, Val Loss: 0.0110

Epoch 5/5: Train Loss: 0.0050, Val Loss: 0.0113

- **Test loss:** 0.0133

- **Test Mean Squared Error:** 0.0140

Method A (Synonym Replacement) improved the model's performance compared to the baseline Task 1 model (MSE 0.0129 vs 0.0136), while Method B (Random Deletion) decreased the performance (MSE 0.0140).

This might have occurred because of the specific nature of the humour detection task. Synonym replacement forces the model to learn the semantic concepts rather than memorizing exact vocabulary. However, random deletion might have removed the punchlines or the context, confusing the model and resulting in lower performance.

Task 3: One way to improve the performance is to perform ensembling of three regressors. Try this and see if it improves the performance of the model from Task 1. Use non-augmented data.

Model: 0

- **Training results:**

Epoch 1/5: Train Loss: 0.0126, Val Loss: 0.0117

Epoch 2/5: Train Loss: 0.0108, Val Loss: 0.0111

Epoch 3/5: Train Loss: 0.0099, Val Loss: 0.0108

Epoch 4/5: Train Loss: 0.0088, Val Loss: 0.0110

Epoch 5/5: Train Loss: 0.0073, Val Loss: 0.0109

- **Test loss:** 0.0121
- **Test Mean Squared Error:** 0.0127

Model: 1

- **Training results:**

Epoch 1/5: Train Loss: 0.0150, Val Loss: 0.0124

Epoch 2/5: Train Loss: 0.0115, Val Loss: 0.0113

Epoch 3/5: Train Loss: 0.0106, Val Loss: 0.0109

Epoch 4/5: Train Loss: 0.0098, Val Loss: 0.0104

Epoch 5/5: Train Loss: 0.0087, Val Loss: 0.0103

- **Test loss:** 0.0116
- **Test Mean Squared Error:** 0.0120

Model: 2

- **Training results:**

Epoch 1/5: Train Loss: 0.0134, Val Loss: 0.0122

Epoch 2/5: Train Loss: 0.0114, Val Loss: 0.0114

Epoch 3/5: Train Loss: 0.0105, Val Loss: 0.0108

Epoch 4/5: Train Loss: 0.0094, Val Loss: 0.0107

Epoch 5/5: Train Loss: 0.0087, Val Loss: 0.0110

- **Test loss:** 0.0127
- **Test Mean Squared Error:** 0.0133

Ensembling the three models resulted in a Test MSE of 0.0135. When compared to the baseline model from Task 1 (Test MSE 0.0136), the ensemble provided a very marginal improvement. On the other hand, individual models within the ensemble (Model 1 with a MSE of 0.0120) outperformed the final averaged ensemble prediction.

Task 4: Build and evaluate the multi-task learning regressor (Task 5) trained simultaneously with the second regression task of offense rating.

Multi-task Learning:

Results:

- **Training results:**

Epoch 1/5: Train Loss: 0.0500, Val Loss: 0.0473

Epoch 2/5: Train Loss: 0.0405, Val Loss: 0.0405

Epoch 3/5: Train Loss: 0.0348, Val Loss: 0.0374

Epoch 4/5: Train Loss: 0.0297, Val Loss: 0.0347

Epoch 5/5: Train Loss: 0.0254, Val Loss: 0.0320

- **Test loss:** 0.0155

- **Test Mean Squared Error:** 0.0155

The Multi-Task Learning model performed worse than the single-task baseline from Task 1 (Test MSE 0.0155 vs 0.0136).

This decline in performance is a classic example of negative transfer. While humour and offense occasionally overlap (for example, in dark humour), they represent fundamentally different data distributions. By forcing the shared hidden layers of the DistilBERT model to optimize for two divergent objectives simultaneously, the network's representational capacity was split. This prevented the network from fully specializing in the primary objective of humour detection, therefore decreasing its overall performance.

PART E: Summarisation and Data Generation

This report explores fine-tuning a T5 transformer model for extreme summarisation and controlled text generation with keywords using the XSum dataset, which contains long BBC articles and their single-sentence summaries.

TASK 1: Summarisation Data Statistics

For this task, I've implemented the functions `calculate_token_counts()` and `token_counts_summary()`.

- `Calculate_token_counts`: Iterates through the dataset, splitting the "document" and "summary" text by whitespace to compute the token counts for each example.

- `Token_counts_summary()`: Uses NumPy to calculate and output the mean and Standard deviation of these token counts for both the training and validation splits.

Results:

- **Train Split:**

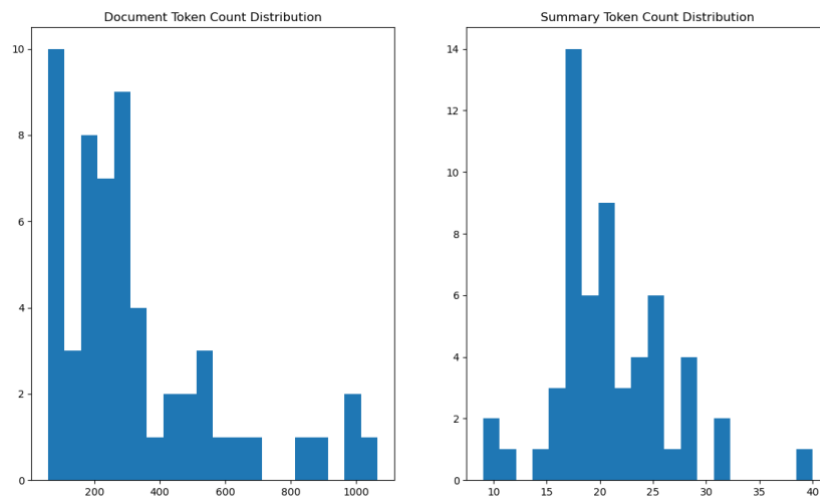
Input Document: Mean= 359.75, Std = 311.73

Target summary: mean = 20.76, std = 4.88

- **Validation Split:**

Input Document: mean = 329.44, std = 247.50

Target summary: mean = 20.82, std = 5.41



Based on the computed statistics and the plotted histograms, there is a significant reduction in text length from the input documents to the target summaries. The original articles (source) are relatively long, averaging between 330 and 360 tokens, whereas the target summaries are heavily compressed, averaging only about 21 tokens.

This is also shown by the standard deviation. The source texts show high variability, with a standard deviation of approximately 250 to 310 tokens. This indicates a wide-spread in article lengths. On the other hand, the summaries have a very low standard deviation of roughly 5 tokens, demonstrating that the target outputs are consistent in length.

The histograms visually confirm these different distributions. The input document plot exhibits a heavily right-skewed distribution, with most documents clustering at the lower end of the word count, accompanied by a long tail of much lengthier articles. On the other hand, the summary distribution is more centralized and symmetric.

TASK 2: Fine-tune and evaluate T5 for the XSum summarisation task

1. Training Performance (5 Epochs)

The model was fine-tuned using a T5-small architecture. The training loss showed some fluctuation due to the small data subset (I used 2% instead of 10%), but the overall performance improved significantly.

Results:

Generic model:

- Final Training Loss: 3.0664
- Validation ROUGE (after 5 epochs):
 - o ROUGE-1: 0.211
 - o ROUGE-2: 0.109
 - o ROUGE-L: 0.140

Whereas the not fine-tuned, the Generic Zero-Shot model:

- ROUGE-1: 0.113
- ROUGE-2: 0.309
- ROUGE-L: 0.068

The results show that fine-tuning is highly efficient for specialized NLP tasks. Even with a small training sample and only five epochs, the fine-tuned model's ROUGE-1 score doubled compared to the generic model.

TASK 3: Build and evaluate a T5-based model for data generation using key-Words

1. Training Results (5 Epochs)

The model was fine-tuned to reconstruct summaries from a randomized set of keywords (20). The loss showed a significant initial drop, followed by some stabilization:

- Epoch 1: Training loss: 4.235673904418945
- Epoch 2: Training loss: 2.649052143096924
- Epoch 3: Training loss: 2.883911609649658
- Epoch 4: Training loss: 3.2128536701202393
- Epoch 5: Training loss: 3.0313334465026855

2. Keyword Generation Output

Input Keywords: *'damage ongoing Hawick disruption Lamington viaduct commercial thoroughfare multi-agency neglected'*

Model Output: *Hawick a disruption Lamington viaduct commercial neglected neglected in Lamington*

The model successfully retained the keywords (Hawick, Lamington, disruption) from the input string, showing that it has prioritized these tokens. However, the output has some repetitions. For example, “Lamington” and “neglected”. The output would need more training data, but with only 2% is already consistent.